SUBHAPREET PATRO
2211CS010547
S-1(76)

MAP ASSIGNMENT-4

1) Creating Success and Error Response Settings for HTTP Listeners in a Mule Application

Understanding Error Response Settings

In Mule applications, *Error Response Settings* determine how the system reacts when something goes wrong in a flow.
Instead of displaying a default or vague error, you can set up Mule to send a specific HTTP response — including the status code, message, and payload — back to the client.

Example:
If an invalid input is sent by a client, you can configure Mule to return a **400 Bad Request** with a descriptive message explaining the issue.

Steps to Configure Success and Error Responses

Step 1: Add an HTTP Listener

- Drag an **HTTP Listener** component to the flow.
- **Path:** /err
- **Purpose:** Receives incoming requests at the /err endpoint.

Step 2: Add a Set Variable Component

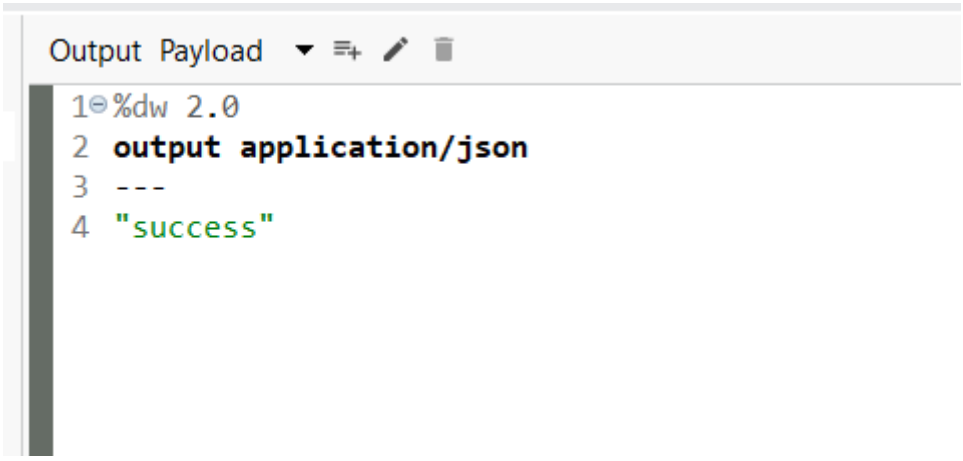
- Place a **Set Variable** right after the listener.
- **Name:** name
- **Value:** abc
- **Purpose:** Stores a variable value that can be reused later in the flow.

Step 3: Add an HTTP Request Component

- Add an **HTTP Request** element after setting the variable.
- **URL:** xyzabc.com
- **Method:** GET
- **Body:** payload
- **Purpose:** Sends the received request data to an external API.

Step 4: Add a Transform Message for Successful Responses

- Insert a **Transform Message** component.
- **Purpose:** Defines the success message (e.g., returning "success") if the API call is successful.

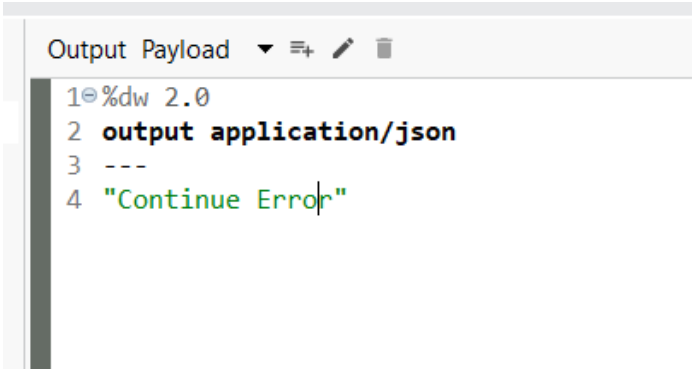


Step 5: Configure Error Handling

- Open the **Error Handling** section in the flow.
- Add an **On Error Continue** block.
- **Error Types:** HTTP:CONNECTIVITY, HTTP:UNAUTHORIZED
- **Purpose:** Captures connection or authentication errors and allows the flow to proceed instead of failing completely.

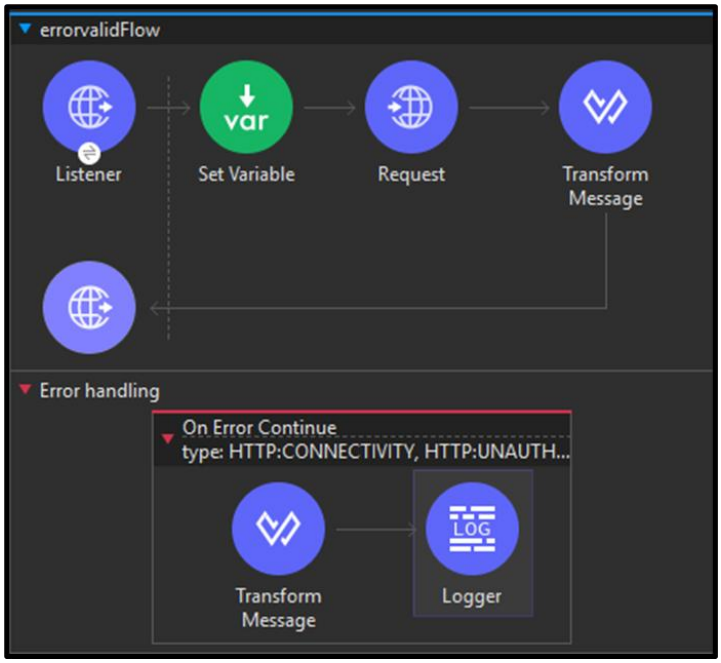
Step 6: Add Transform Message for Error Response

- Inside the error handler, add another **Transform Message** component.
- **Purpose:** Returns a custom message like "continue error" when connection or authorization problems occur.



Step 7: Add a Logger

- Place a **Logger** after the error message transformation.
- **Message:** #[payload]
- **Purpose:** Logs the custom error message in the Anypoint Studio console.



Step 8: Test the Flow

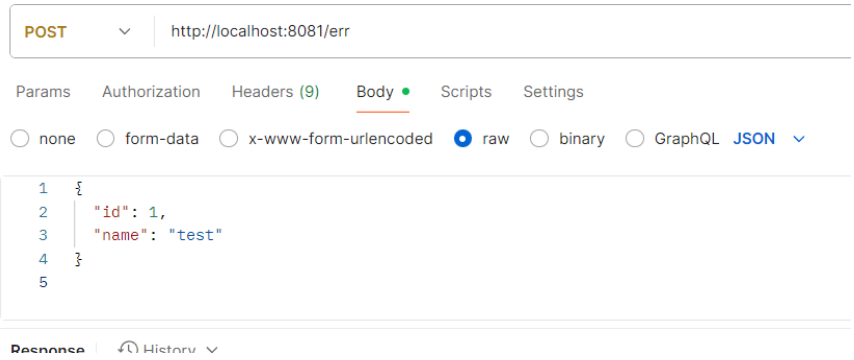
Run the Mule app and test using **Postman** or another REST client.

- **Method:** POST or GET
- **URL:** http://localhost:8081/err
- **Sample Body:**

```
{
  "id": 1,
  "name": "test"
}
```

Expected Output:

- If the external API works: "success"
- If the API fails or is unauthorized: "continue error"



2) DataWeave Selectors

DataWeave selectors help navigate objects and arrays to extract values that match certain keys or positions. They always operate relative to a *context* — this can be a variable, an object, an array, or the output of another function.

Selector Type	Syntax	Description	Output Type
Single-value	.keyName	Retrieves the value of a specific key	Value
Multi-value	.*keyName	Returns all values matching a key	Array
Descendants	..keyName	Returns all nested key matches	Array
Index	[index]	Retrieves an element at a specific array position	Value
Range	[start to end]	Returns multiple values in a specified range	Array

Single-Value Selector

Example:

Input:

```
{
  "name": "Ana",
  "age": 29
}
```

DataWeave Script:

```
%dw 2.0
output json
---
payload.age
```

Output:

29

You can also use square brackets for dynamic key references:

```
%dw 2.0
output json
---
{
  fixed: payload.age,
  dynamic: payload[payload.dynamicKey]
}
```

Index Selector

Used for accessing elements in an array:

```
%dw 2.0
output json
---
payload[1]
```

Input: ["prod", "qa", "dev"]
Output: "qa"

Negative indexes count from the end, e.g., payload[-1] gives the last element.

Range Selector

Used to fetch a range of elements:

```
%dw 2.0

output json

---

payload[0 to 1]
```

Output: ["prod", "qa"]

Multi-Value Selector

Fetches all values for a key that appears multiple times:

```
%dw 2.0

output json

---

payload.*name
```

Output: ["Emilia", "Isobel", "Euphemia", "Rose"]

Descendants Selector

Retrieves all values of a key regardless of their depth in the data structure:

```
%dw 2.0

output json

---

payload..echo
```

Output:

[{ "value": "Hello there!"}, "Getting details...", "Success!"]

3) DataWeave Functions

SUBHAPREET PATRO
2211CS010547
S-1(76)

Functions in DataWeave let you encapsulate logic for reuse. They can be *named* or *anonymous (lambdas)* and are crucial for modular scripts.

Named Functions

A named function is declared with the fun keyword:

```
fun add(a, b) = a + b

---

add(1, 2)
```

Output: 3

Lambda Functions

Anonymous functions without names, written as:

```
(x, y) -> x + y
```

They are useful when passed as arguments or returned from other functions.

Higher-Order Functions

Functions that accept other functions as arguments or return them.
Example using filter:

```
%dw 2.0

output json

fun isOdd(n) = (n mod 2) == 1

var numbers = (1 to 5)

---

filter(numbers, (n) -> isOdd(n))
```

Output: [1, 3, 5]

Infix Notation

A concise way to call two-argument functions:

```
%dw 2.0

output json

var numbers = (1 to 5)
```

---

numbers filter ((n) -> (n mod 2) == 1)

Syntax

Special placeholders simplify lambda expressions:

%dw 2.0

output json

var numbers = (1 to 5)

---

numbers filter ($ mod 2 == 1)

Output: [1, 3, 5]

These can also be chained to access object properties:

payload filter $.price > 5

Output:

```
[
  { "id": 2, "item": "steak", "price": 15.00 }
]
```